<u>Systèmes d'Exploitation -</u> Les Interblocages

Introduction

Les interblocages (ou deadlocks) sont des situations problématiques dans les systèmes d'exploitation où plusieurs processus se bloquent mutuellement, empêchant ainsi toute progression. Pour comprendre ce phénomène, prenons un exemple simple : imaginez deux trains arrivant à un croisement. Si les deux trains s'arrêtent et attendent que l'autre parte avant de redémarrer, aucun des deux ne pourra avancer. C'est exactement ce qui se passe dans un système d'exploitation lorsqu'un interblocage survient.

1. Théorie des Interblocages

Modélisation des Ressources

Dans un système d'exploitation, les ressources sont des éléments essentiels que les processus utilisent pour fonctionner. Ces ressources peuvent être de différents types : CPU, mémoire, disque, imprimantes, etc. Chaque type de ressource peut avoir une ou plusieurs instances. Par exemple, il peut y avoir plusieurs imprimantes disponibles dans un système.
Les ressources peuvent être classées en deux catégories :

- **Ressources réquisitionnables** : Ces ressources peuvent être récupérées pendant leur utilisation sans causer de problèmes. Par exemple, la mémoire peut être réallouée à un autre processus si nécessaire.
- **Ressources non réquisitionnables** : Ces ressources ne peuvent pas être récupérées sans causer des problèmes. Par exemple, une imprimante en cours d'utilisation ne peut pas être réquisitionnée sans interrompre l'impression en cours.

Les interblocages concernent principalement les ressources non réquisitionnables.

Conditions Nécessaires pour un Interblocage

Pour qu'un interblocage se produise, quatre conditions doivent être réunies :

1. **Exclusion mutuelle** : Une ressource ne peut être utilisée que par un seul processus à la fois.

2. **Occupation et attente** : Un processus qui occupe une ressource peut en attendre une autre.
3. **Pas de réquisition** : Une ressource ne peut être libérée que par le processus qui l'occupe.
4. **Attente circulaire** : Il existe un cycle de processus où chaque processus attend une ressource occupée par un autre processus.

Ces conditions sont nécessaires mais pas suffisantes. Cela signifie que si ces conditions sont réunies, un interblocage peut se produire, mais il n'est pas certain qu'il se produise.

■ Exemple d'interbloquage

Processus 1 :	**Processus 2 :**
`wait (sem1);`	`wait (sem2);`
`wait (sem2);`	`wait (sem1);`

Explication :
Ce scénario représente un exemple classique d'**interblocage** (ou **deadlock**) entre deux processus qui utilisent des sémaphores pour synchroniser leur accès à des ressources partagées. Voici une explication détaillée de ce qui se passe :

Contexte :

- **Processus 1** et **Processus 2** sont deux processus concurrents qui tentent d'accéder à deux ressources protégées par des sémaphores : **sem1** et **sem2**.
- Les sémaphores sont des mécanismes de synchronisation utilisés pour gérer l'accès à des ressources partagées. Un sémaphore peut être vu comme un verrou : un processus doit acquérir le verrou (via wait) avant d'accéder à la ressource, et le libérer (via signal) une fois qu'il a terminé.

Séquence d'actions :

1. **Processus 1** exécute :

    - wait(sem1); : Il acquiert le sémaphore **sem1** et continue son exécution.
    - wait(sem2); : Il tente d'acquérir le sémaphore **sem2**, mais si **sem2** est déjà pris par un autre processus (ici, **Processus 2**), **Processus 1** est bloqué en attendant que **sem2** soit libéré.
2. **Processus 2** exécute :
    - wait(sem2); : Il acquiert le sémaphore **sem2** et continue son exécution.
    - wait(sem1); : Il tente d'acquérir le sémaphore **sem1**, mais si **sem1** est déjà pris par un autre processus (ici, **Processus 1**), **Processus 2** est bloqué en attendant que **sem1** soit libéré.

Résultat : Interblocage

- **Processus 1** attend que **Processus 2** libère **sem2**.
- **Processus 2** attend que **Processus 1** libère **sem1**.
- Aucun des deux processus ne peut continuer, car chacun détient un sémaphore dont l'autre a besoin. Cela crée une situation d'**interblocage** : les deux processus sont bloqués indéfiniment.

Conditions nécessaires pour l'interblocage :

Cet exemple illustre les quatre conditions nécessaires pour qu'un interblocage se produise :

1. **Exclusion mutuelle** : Les sémaphores **sem1** et **sem2** ne peuvent être détenus que par un seul processus à la fois.
2. **Occupation et attente** : **Processus 1** détient **sem1** et attend **sem2**, tandis que **Processus 2** détient **sem2** et attend **sem1**.
3. **Pas de réquisition** : Les sémaphores ne peuvent pas être réquisitionnés ; un processus doit libérer un sémaphore de manière volontaire.
4. **Attente circulaire** : Il y a un cycle d'attente entre les deux processus.

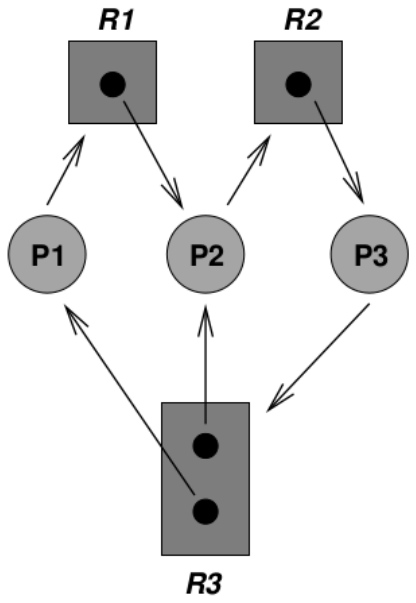
Graphe d'Allocation de Ressources

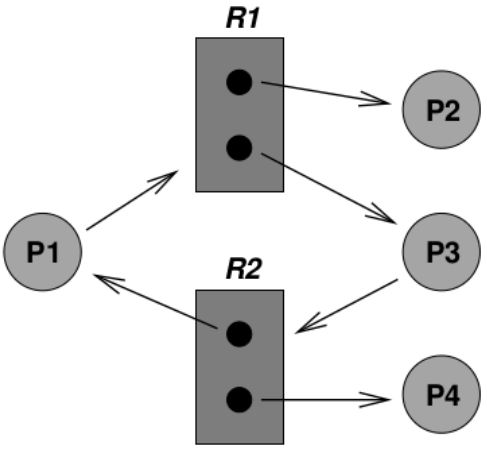
Pour visualiser les interblocages, on utilise un graphe d'allocation de ressources. Ce graphe est composé de deux types de nœuds : les processus (P)

et les ressources (R). Les arcs représentent les requêtes et les allocations de ressources.

- Un arc de requête $P_i \rightarrow R_j$ indique que le processus P_i demande la ressource R_j.
- Un arc d'affectation $R_j \rightarrow P_i$ indique que la ressource R_j est allouée au processus P_i.

Un interblocage se produit si le graphe contient un cycle. Si chaque ressource n'a qu'une seule instance, la présence d'un cycle est suffisante pour qu'un interblocage se produise.





2. Traitement des Interblocages

Il existe plusieurs approches pour traiter les interblocages :

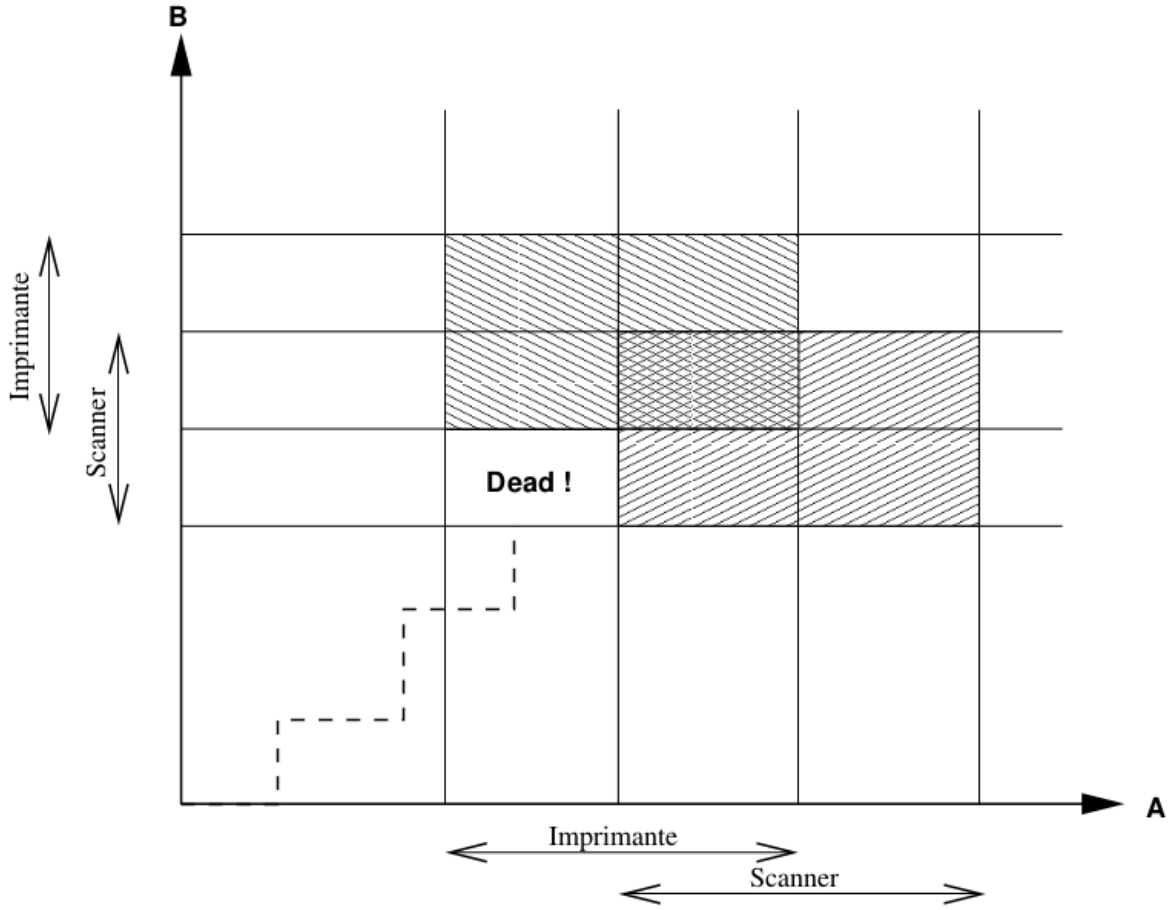
- **Prévention** : Empêcher l'une des conditions nécessaires à l'interblocage.
- **Évitement** : Utiliser des informations sur les ressources pour éviter les interblocages.
- **Détection** : Détecter les interblocages lorsqu'ils se produisent.
- **Correction** : Corriger les interblocages une fois détectés.

Prévention des Interblocages

La prévention consiste à empêcher l'une des quatre conditions nécessaires à l'interblocage. Par exemple :

- **Exclusion mutuelle** : Utiliser des techniques comme le spooling pour éviter l'exclusion mutuelle.

- **Occupation et attente** : Forcer les processus à demander toutes leurs ressources avant de commencer leur exécution.
- **Pas de réquisition** : Si un processus demande une ressource occupée, libérer toutes ses ressources et le mettre en attente.
- **Attente circulaire** : Imposer un ordre total sur l'allocation des ressources.



- Expliquer l'interblocage
- Comment peut-on prévenir ?

Évitement des Interblocages

L'évitement consiste à s'assurer que le système reste dans un "état sain", c'est-à-dire un état où il est possible de trouver un ordonnancement des processus qui garantit que tous les processus pourront terminer leur exécution. Pour cela, on utilise des algorithmes comme l'algorithme du banquier, qui vérifie si l'allocation d'une ressource conduirait à un état malsain.

Algorithme de requête de ressources

(1) **si** request [P_i] <= need [P_i]**:**
(2) aller en 5.
(3) **sinon:**
(4) erreur (demande supérieure à la réclamation initiale).
(5) **si** request [P_i] <= free**:**
(6) aller en 9.
(7) **sinon:**
(8) bloquer P_i.
(9) Faire tourner l'algorithme de détermination d'état avec :
(10) free $\leftarrow$ free - request [P_i]
(11) used [P_i] $\leftarrow$ used [P_i] + request [P_i].
(12) Si l'état calculé est sain, faire la mise à jour.

Algorithme de détermination d'état

(1) Chercher P_i non marqué tel que request [P_i] <= free.
(2) Si P_i n'existe pas, aller en 6.
(3) free $\leftarrow$ free + used [P_i].
(4) Marquer P_i.
(5) Aller en 1.
(6) S'il reste un processus non marqué, alors le système est en état malsain.

Avec le schéma suivant, en suivant l'algorithme du banquier vérifions si chacun des systèmes est dans un état d'interblocage. Ci-après les étapes :

- Pour chaque état, calculer les besoins (Need) de chaque processus : Need = Max - Used
- Vérifier s'il existe une séquence sûre d'allocation des ressources
- Pour qu'il n'y ait pas d'interblocage, il faut qu'il existe au moins un processus dont les besoins peuvent être satisfaits avec la mémoire libre disponible.

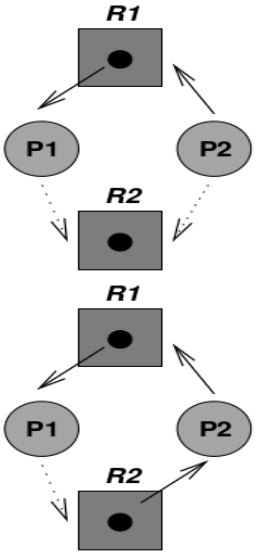
Proc / Used / Max :

A	3	9	3	9	3	9	3	9	3	9
B	2	4	4	4	0	-	0	-	0	-
C	2	7	2	7	2	7	7	7	0	-
∅	3		1		5		0		7	

A	3	9	4	9	4	9	4	9
B	2	4	2	4	4	4	0	-
C	2	7	2	7	2	7	2	7
∅	3		2		0		4	

<u>ALGORITHME DU GRAPHE D'ALLOCATION (Si chaque ressource n'existe qu'en une seule instance)</u>

- **arc de réclamation** ($P_i \rightarrow R_j$) : indique que P_i demandera R_j dans le futur.
- Réclamation des ressources *a priori*.
- Une libération de ressource transforme un arc d'affectation en arc de réclamation.
- La transformation d'un arc de réclamation en arc d'affectation ne doit pas introduire de cycle (« état malsain »).



Détection et Correction des Interblocages

Si les interblocages ne sont pas évités, il faut les détecter et les corriger. La détection peut se faire en utilisant un graphe d'attente, qui est une version simplifiée du graphe d'allocation de ressources. Si un cycle est détecté, un interblocage est présent.

Pour corriger un interblocage, on peut :

- **Réquisitionner des ressources** : Libérer des ressources occupées par certains processus pour permettre à d'autres de continuer.

- **Terminer des processus** : Arrêter un ou plusieurs processus impliqués dans l'interblocage.

Algorithme général de détection

(1) Chercher P_i non marqué tel que request [P_i] <= free.
(2) **si** P_i n'existe pas**:**
(3) aller en 7.
(4) free $\leftarrow$ free + used [P_i].
(5) Marquer P_i.
(6) Aller en 1.
(7) **si** il reste un processus non marqué**:**
(8) il est en situation d'interbloquage.

Systèmes d'Exploitation - La Gestion de la Mémoire

Introduction

La gestion de la mémoire est une fonction essentielle des systèmes d'exploitation. Elle consiste à allouer et libérer de la mémoire pour les processus, tout en optimisant l'utilisation de la mémoire physique.

1. Généralités sur la Gestion de la Mémoire

Niveaux d'Adressage

Edit

Dans un système informatique, il existe plusieurs niveaux d'adressage essentiels pour la gestion de la mémoire :

- **Adresse symbolique** :
  - Utilisée dans le code source du programme
  - Constituée de noms de variables, fonctions, étiquettes
  - Compréhensible par le programmeur
  - Exemple : `int compteur = 0;` où "compteur" est l'adresse symbolique
  - Convertie en adresse logique lors de la compilation ou de l'édition de liens
- **Adresse logique (ou virtuelle)** :
  - Générée par le CPU lors de l'exécution du programme
  - Composée d'un numéro de page/segment et d'un décalage (offset)
  - Indépendante de l'emplacement physique réel
  - Permet l'isolation des processus
  - Exemple : `0x00400000` (adresse de base) + `0x1234` (offset)
  - Chaque processus a son propre espace d'adressage logique
- **Adresse physique** :
  - L'emplacement réel en mémoire RAM où les données sont stockées
  - Représente la position exacte dans la mémoire physique
  - Unique pour chaque emplacement mémoire
  - Exemple : `0x7FFF1234` représentant un emplacement précis dans la RAM
  - Invisible aux programmes utilisateurs
  - Gérée par le système d'exploitation
- **MMU (Memory Management Unit)** :
  - Composant matériel intégré au processeur
  - Traduit automatiquement les adresses logiques en adresses physiques
  - Utilise une table des pages pour la traduction
  - Gère la pagination et la segmentation
  - Permet la mémoire virtuelle
  - Exemple de traduction :
    - Adresse logique : Page 5, offset 123

        - Table des pages : Page 5 → Frame 10
        - Adresse physique résultante : Frame 10, offset 123
    - Inclut un TLB (Translation Lookaside Buffer) pour accélérer les traductions

Ce système d'adressage multiniveau permet :

- L'indépendance des programmes vis-à-vis de la mémoire physique
- La protection de la mémoire entre processus
- L'optimisation de l'utilisation de la mémoire
- La mise en œuvre de la mémoire virtuelle

EXEMPLE :

Programme en C avec une variable globale

```
int compteur = 42;
```

Étape 1 : Adresse Symbolique → Adresse Logique

- Adresse symbolique : "compteur"
- Lors de la compilation et l'édition de liens :
    - La variable est placée dans le segment de données
    - Soit une adresse logique : 0x00500000 (segment) + 0x100 (offset)
    - Adresse logique résultante : 0x00500100

Étape 2 : Adresse Logique → Traduction par MMU

- L'adresse logique 0x00500100 est divisée en :
    - Numéro de page : 0x005 (bits de poids fort)
    - Offset dans la page : 0x100 (bits de poids faible)
- Consultation de la table des pages :
    - Page 0x005 → Frame physique 0x8A2

Étape 3 : Construction de l'Adresse Physique

- Frame physique : 0x8A2 (base)
- Offset : 0x100 (reste inchangé)
- Adresse physique finale : 0x8A2100

Donc : "compteur" (symbolique) → 0x00500100 (logique) → 0x8A2100 (physique)

Pratique adressage mémoire : Traduction d'adresses dans un système de mémoire virtuelle

Énoncé : Soit un système avec les caractéristiques suivantes :

- Taille des pages : 1024 octets (1 Ko)
- Table des pages du processus P1 :

Page virtuelle	Frame physique	Bit validité
0	3	1
1	7	1
2	4	1
3	-	0
4	8	1

Questions :

1. Pour les adresses logiques suivantes, calculez l'adresse physique correspondante : a) 0x845 b) 0x1050 c) 0x2200
2. Pour chaque adresse :
    - Identifiez le numéro de page virtuelle et l'offset
    - Utilisez la table des pages pour trouver le frame physique
    - Calculez l'adresse physique finale
    - Indiquez si l'accès causera un défaut de page
3. L'adresse physique 0x2845 correspond à quelle adresse logique ?

Solution : Je vous laisse réfléchir, mais voici comment procéder :

1. Convertir l'adresse logique en binaire
2. Séparer les bits de page (10 bits pour 1Ko) des bits d'offset
3. Utiliser la table des pages
4. Combiner le frame avec l'offset

Liaison d'Adresse

La liaison d'adresse est le processus qui transforme les adresses symboliques en adresses physiques. Cela peut se faire à différents moments :

- **Compilation** : Le code est compilé en code absolu, avec des adresses fixes.
- **Chargement** : Le code est translatable, et les adresses sont ajustées lors du chargement en mémoire.
- **Exécution** : Le code est dynamique, et les adresses peuvent varier pendant l'exécution.

Comparaison des trois approches :

Caractéristique	Compilation	Chargement	Exécution
Flexibilité	Faible	Moyenne	Élevée
Performance	Excellente	Bonne	Moyenne
Complexité	Simple	Moyenne	Complexe
Usage typique	Systèmes embarqués	Programmes simples	OS modernes

Le choix du type de liaison dépend de :

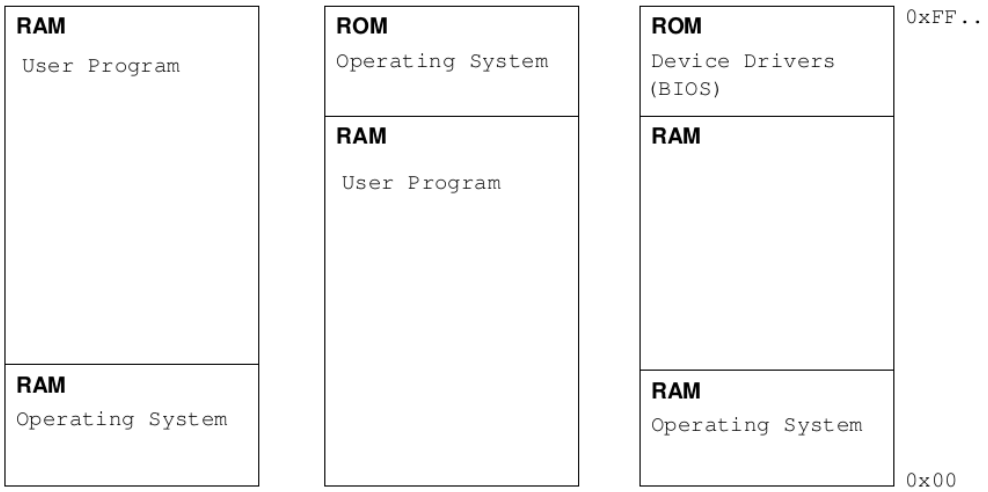
- Les besoins en performance
- La flexibilité requise
- Le type de système
- Les contraintes matérielles
- Les besoins en sécurité

2. Techniques de Gestion de la Mémoire

Monoprogrammation

Dans un système monoprogrammé, un seul processus est exécuté à la fois. La mémoire est divisée en deux parties : une pour le système d'exploitation et une pour le programme utilisateur. Cette approche est simple mais inefficace, car elle ne permet pas d'utiliser pleinement les ressources du système.

<u>Monoprogrammation (Mainframes, Palm et systèmes embarqués, MS-DOS)</u>

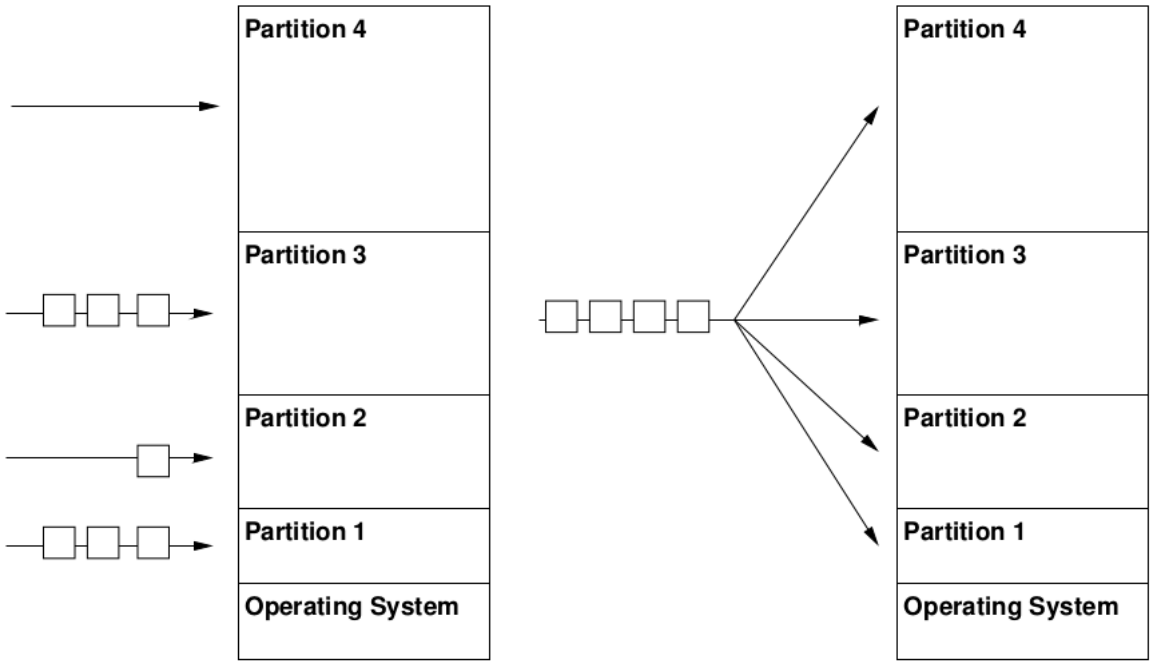


Multiprogrammation

Dans un système multiprogrammé, plusieurs processus sont exécutés simultanément. La mémoire est divisée en partitions, et chaque processus est alloué à une partition. Il existe deux types de partitions :

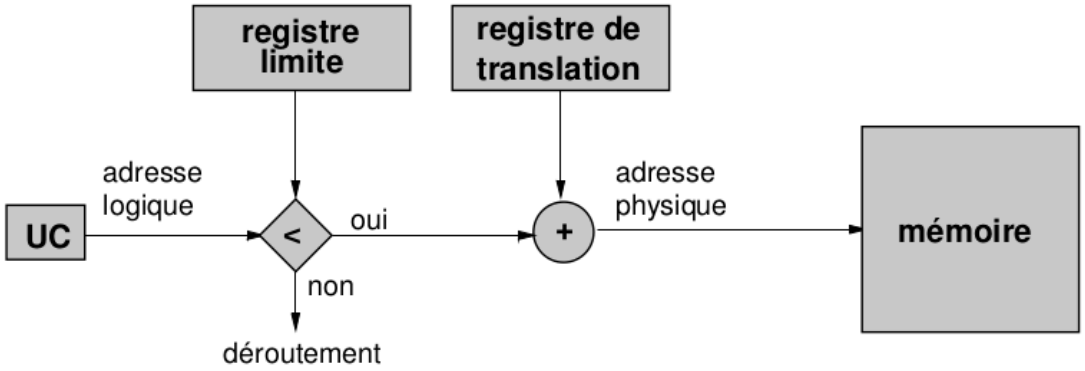
- **Partitions fixes** : La mémoire est divisée en partitions de taille fixe.
- **Partitions dynamiques** : La mémoire est allouée dynamiquement en fonction des besoins des processus.

Multiprogrammation & partitions fixes (« OS/MFT » sur OS/360)



Localisation et protection

- **Par MMU :** (CDC 6600, Intel 8088)



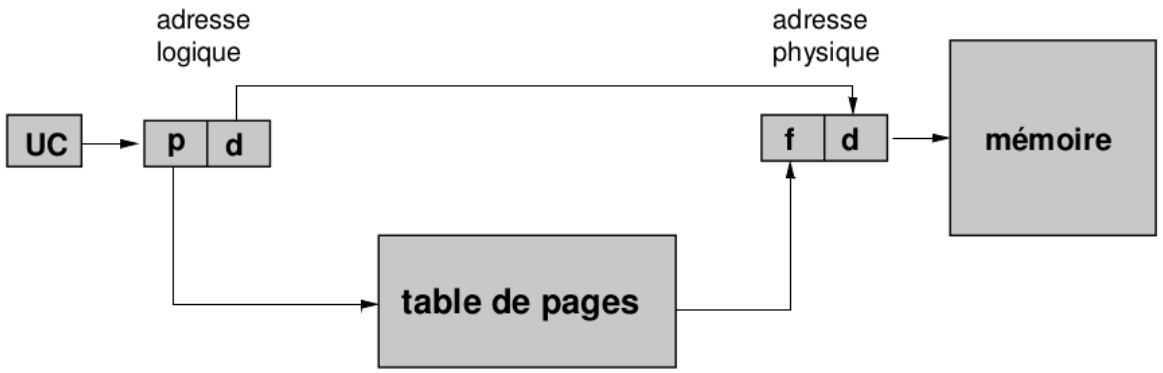
Pagination

La pagination est une technique de gestion de la mémoire utilisée dans les systèmes d'exploitation pour gérer efficacement la mémoire physique et virtuelle. Voici une explication détaillée de ce concept avec des exemples :

1. **Division de la mémoire** :
   - **Mémoire physique** : La mémoire physique (RAM) est divisée en petits blocs de taille fixe appelés **cadres de page**. Par exemple, si la taille d'une page est de 4 Ko, alors chaque cadre de page dans la mémoire physique aura également une taille de 4 Ko.
   - **Mémoire logique** : La mémoire logique (ou virtuelle) vue par un processus est également divisée en blocs de même taille, appelés **pages**. Ainsi, si un processus a besoin de 16 Ko de mémoire, il utilisera 4 pages logiques de 4 Ko chacune.
2. **Table de pages** :
   - Chaque processus a une **table de pages** qui maintient une correspondance entre les pages logiques et les cadres de page physiques. Par exemple, si la page logique 1 d'un processus est mappée sur le cadre de page physique 3, cette information est stockée dans la table de pages.
   - Lorsque le processus accède à une adresse mémoire, le système d'exploitation utilise la table de pages pour traduire l'adresse logique en adresse physique.
3. **Fragmentation** :
   - **Fragmentation externe** : La pagination réduit la fragmentation externe, qui se produit lorsque la mémoire libre est dispersée en petits blocs non contigus. Comme les pages sont de taille fixe, elles peuvent être allouées n'importe où dans la mémoire physique, ce qui minimise ce type de fragmentation.
   - **Fragmentation interne** : Cependant, la pagination peut entraîner une fragmentation interne. Cela se produit lorsque la mémoire allouée à un processus n'est pas complètement utilisée. Par exemple, si un processus n'a besoin que de 5 Ko de mémoire, il devra tout de même utiliser deux pages de 4 Ko, laissant 3 Ko inutilisés dans la deuxième page.
4. **TLB (Translation Look-aside Buffer)** :
   - Pour accélérer la traduction des adresses, les systèmes utilisent une **TLB**, qui est un cache spécial pour les entrées de la table de pages. La TLB stocke les traductions d'adresses récemment utilisées.
   - Par exemple, si un processus accède fréquemment à une adresse mémoire, la traduction de cette adresse sera probablement dans la

TLB, ce qui permet une traduction rapide sans avoir à consulter la table de pages en mémoire principale.

M MU pour la pagination



Segmentation

La segmentation est une technique de gestion de la mémoire qui divise la mémoire en segments de taille variable, contrairement à la pagination où la mémoire est divisée en blocs de taille fixe. Chaque segment représente une unité logique distincte, ce qui permet une organisation plus intuitive de la mémoire en fonction des besoins des programmes. Par exemple, un programme peut avoir un segment pour le code exécutable, un autre pour les données, et un autre pour la pile.

Exemples de segments :

1. Segment de code : Contient les instructions du programme. Par exemple, si vous avez un programme en C, le code compilé serait stocké dans ce segment.
2. Segment de données : Contient les variables globales et statiques. Par exemple, si vous avez une variable globale int x = 10;, elle serait stockée ici.
3. Segment de pile : Utilisé pour gérer les appels de fonctions et les variables locales. Par exemple, lors d'un appel de fonction, les paramètres et les variables locales sont poussés sur la pile.

Table de segments :

Chaque processus possède une table de segments qui mappe les segments logiques (vus par le programme) aux segments physiques dans la mémoire réelle. Par

exemple, si un programme a besoin d'accéder à son segment de code, la table de segments indique où ce segment est physiquement situé en mémoire.

Avantages :

- Partage de mémoire : Les segments peuvent être partagés entre différents processus. Par exemple, plusieurs instances d'un même programme peuvent partager le même segment de code.
- Protection : Chaque segment peut avoir des permissions spécifiques (lecture, écriture, exécution), ce qui améliore la sécurité. Par exemple, le segment de code peut être marqué comme exécutable mais non modifiable.

Inconvénients :

- Fragmentation externe : Comme les segments sont de taille variable, il peut y avoir des espaces libres entre eux qui ne sont pas utilisables pour d'autres segments. Par exemple, si un segment de 1 Ko est libéré, mais que le prochain segment nécessite 2 Ko, cet espace de 1 Ko reste inutilisé, ce qui peut entraîner une fragmentation externe.

3. Pagination à Plusieurs Niveaux

Dans les systèmes modernes, la pagination peut être hiérarchisée en plusieurs niveaux pour gérer de grandes tables de pages. Par exemple, sur un système 32 bits avec des pages de 4 Ko, une table de pages peut contenir un million d'entrées, ce qui nécessite une pagination à plusieurs niveaux pour éviter de charger toute la table en mémoire.

5. Table de Pages Inversée

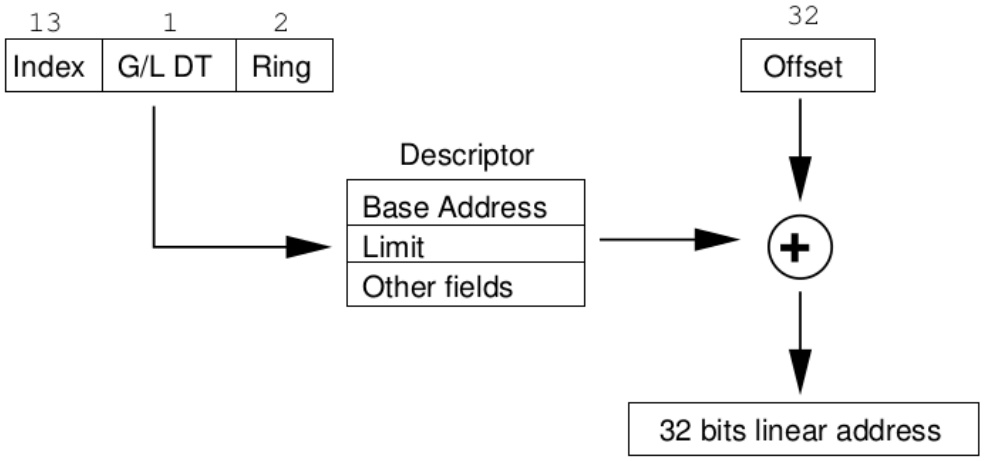
Une autre solution pour gérer de grandes tables de pages est la table de pages inversée. Au lieu d'avoir une table de pages par processus, il y a une seule table de pages pour tout le système, indexée par les cadres de page. Cette approche réduit la taille des tables de pages, mais rend la traduction d'adresses plus complexe.

6. Segmentation et Pagination Combinées

Certains systèmes, comme l'Intel Pentium, combinent la segmentation et la pagination. Chaque processus a plusieurs segments, et chaque segment est paginé. Cela permet une gestion plus flexible de la mémoire, avec des avantages en termes de protection et de partage.

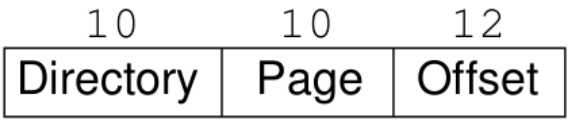
Segmentation et Pagination (Exemple : Intel Pentium)

- 16K segments de 4 Go par processus, dont :
  - 8K segments privés (une table de descripteurs locale « LDT » par processus)
  - 8K segments partageables (une table de descripteurs globale « GDT)
- Sélecteur de segment (16 bits) + offset (32 bits) $\Longrightarrow$ « Adresse linéaire » (32 bits)



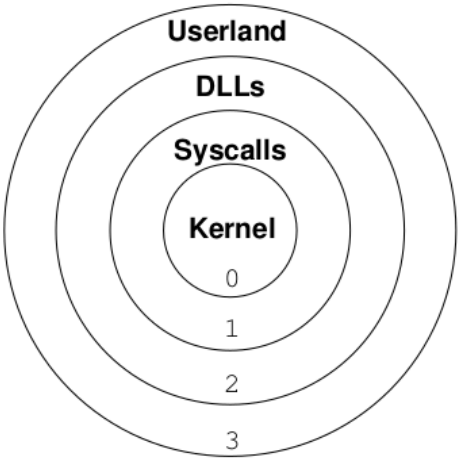
Pagination :

- Pagination à 2 niveaux.
- Pages de 4Ko.



Protection :

- 4 anneaux différents.
- Progression en profondeur par portes / guichets.



Conclusion

Les interblocages et la gestion de la mémoire sont des concepts fondamentaux en architecture des systèmes d'exploitation. Comprendre ces concepts est essentiel pour concevoir des systèmes efficaces et robustes. En maîtrisant les techniques de prévention, d'évitement, de détection et de correction des interblocages, ainsi que les différentes méthodes de gestion de la mémoire, vous serez en mesure de concevoir des systèmes d'exploitation performants et fiables.